

Diseño de la Arquitectura del Sistema

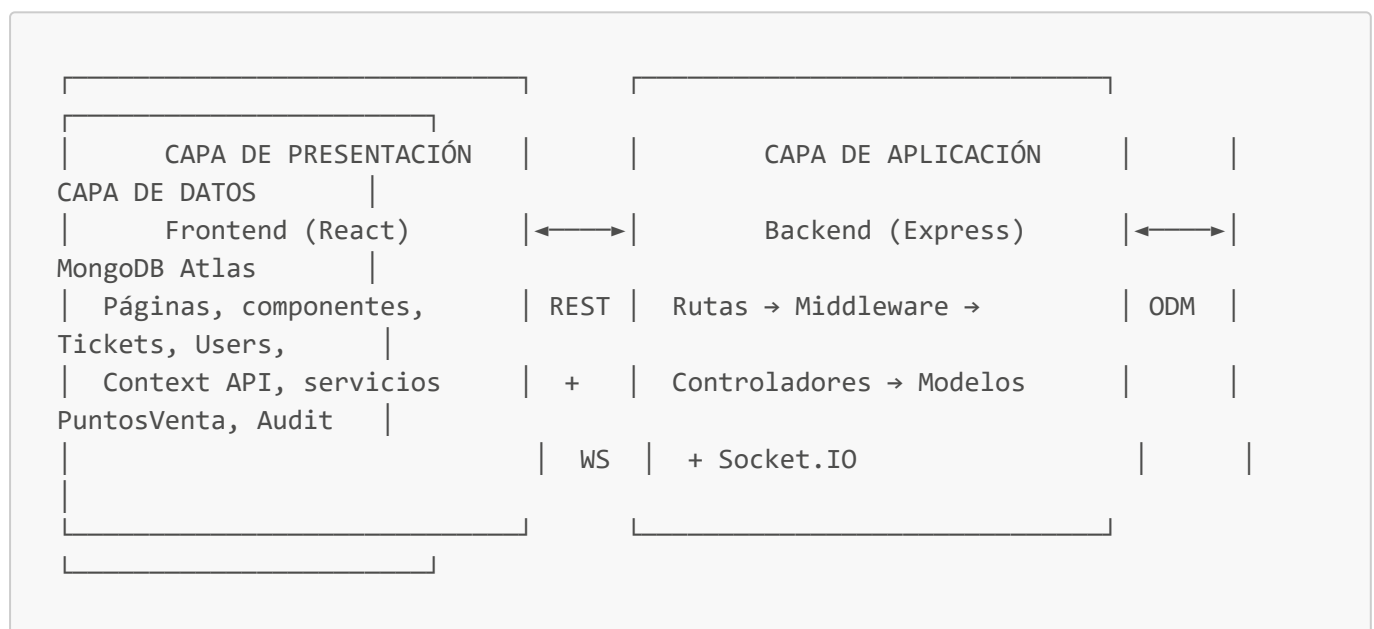
Actividad N°: 11 **Fecha:** 15/06/2026 **Horario:** 14:00 a 20:00 **Tutor Empresarial:** Miguel Vivanco **Pasante:** Gabriel

1. Objetivo del día

Diseñar la arquitectura general del sistema a partir del stack y las decisiones técnicas aprobadas en la Semana 2, definiendo capas, patrones y flujo de comunicación entre frontend, backend y base de datos.

2. Estilo arquitectónico

Se adopta una arquitectura **cliente-servidor de 3 capas**, con una API REST como contrato principal y un canal complementario de WebSocket para eventos en tiempo real:



3. Patrón interno del backend (estilo MVC adaptado)

- **Routes** (`src/routes/*.js`): definen los endpoints y encadenan los middlewares correspondientes.
- **Middleware** (`src/middleware/*.js`): resuelven responsabilidades transversales antes de llegar al controlador:
 - `auth`: valida el JWT y adjunta `req.user`.
 - `authorize(...roles)`: valida permisos por rol.
 - `selectCollection`: adjunta `req.TicketModel` según la colección activa.
 - `auditLogger`: registra automáticamente ciertas operaciones en el log de auditoría.
- **Controllers** (`src/controllers/*.js`): contienen la lógica de negocio (validaciones, reglas, respuesta HTTP).
- **Models** (`src/models/*.js`): esquemas Mongoose que representan las colecciones de MongoDB.

Este patrón permite que cada endpoint sea una composición explícita de middlewares reutilizables, por ejemplo:

```
router.post('/:id/canje', auth, authorize('jefe', 'staff'), auditLogger('canje'),
  canjeTicket);
```

4. Patrón en el frontend

- **Pages:** una página por vista funcional (Dashboard, TicketsPage, UsersPage, AuditPage, PuntosVenta, Login, ChangePassword).
- **Components:** elementos reutilizables entre páginas (Layout, Navigation, ProtectedRoute, RoleBasedRedirect, Unauthorized).
- **Context API (AuthContext):** fuente única de verdad del usuario autenticado y su token, evita prop-drilling.
- **Services:** capa de acceso a datos (`api.js` para REST vía Axios, `socket.js` como singleton de conexión WebSocket).

5. Comunicación en tiempo real (diseño de salas)

Se diseñan **salas (rooms) de Socket.IO** para aislar la información según el contexto del usuario:

```
punto-venta-{idPuntoVenta} → usuarios Jefe siguiendo un punto de venta
específico
staff-{nombrePuntoTrabajo} → usuarios Staff de un punto de trabajo específico
```

Cuando ocurre un canje o impresión, el backend emite el evento `ticket-updated` únicamente a las salas relevantes, evitando que un usuario reciba datos de puntos de venta ajenos (principio de necesidad de conocimiento, alineado con RNF-03).

6. Alineación con requerimientos no funcionales

Decisión de arquitectura	RNF que soporta
Middleware <code>auth/authorize</code> en cada ruta sensible	RNF-01, RNF-03
Salas de Socket.IO aisladas por punto de venta/trabajo	RNF-03 (control de acceso), RF-13
Índices en MongoDB sobre campos de búsqueda frecuente	RNF-07 (rendimiento)
Middleware de auditoría desacoplado del controlador	RNF-08 (trazabilidad) sin acoplar lógica de negocio
Despliegue independiente de frontend/backend	RNF-10

7. Conclusiones del día

Se define una arquitectura de 3 capas con un patrón de middlewares componibles en el backend y una separación clara de responsabilidades en el frontend, que servirá de base para el diseño detallado de la base de datos (día siguiente) y de los módulos de software.

Observaciones: Sin observaciones.